

Bayesian Orchestration of Tool-Augmented Agents

Sequential Monte Carlo over Action Trajectories,
with Application to Retrieval-Augmented Generation

Angshul Majumdar

Department of Electronics and Communication Engineering
Indraprastha Institute of Information Technology Delhi
`angshul@iiitd.ac.in`

July 29, 2026

Abstract

Tool-augmented language model agents select actions greedily and recover from failure by retrying. Both behaviours follow from a single design decision: the agent maintains exactly one execution trajectory and discards the alternatives. Evidence that arrives later—a failed validation, a contradicting source—then has no surviving branch to promote, and a retry redraws from the same proposal distribution rather than conditioning on what was observed. It is a draw from the prior, not a posterior update.

We formulate agent execution as sequential inference over action trajectories under a finite-horizon generative model, and show that three familiar behaviours—deterministic greedy selection, probabilistic forward sampling, and Bayesian sequential Monte Carlo (SMC)—are inference strategies over *one* model rather than distinct architectures. Making the inference regime a runtime parameter permits controlled comparison: identical tools, proposals, and likelihoods run under all three, so any difference in outcome is attributable to inference alone.

Each particle additionally carries private Beta–Bernoulli beliefs over tool reliability, updated online with *retroactive credit assignment*: a checker’s verdict at step t revises the standing of whichever tool produced the claim it examined at step $t - 1$. We further show that answer selection must be governed by validation status rather than recency, and that pooling posterior mass by answer rather than by trajectory yields a calibration signal that identifies when the evidence does not settle a question—at which point the agent asks rather than guesses.

Empirically, on stochastically unreliable tools the method converts a source correct one call in ten into a system correct nine times in ten. On three retrieval-augmented generation failure modes—coreference, staleness under vocabulary drift, and index noise—evaluated against BM25, TF-IDF/cosine, and LSA, it achieves 1.000, 0.903, and 1.000 against a conventional single-pass pipeline’s two deterministic failures and 0.420. We also report where the method does *not* help: a correctly engineered conditional graph is competitive and cheaper when the failure mode is known in advance, and a confidently misspecified prior costs more than it costs a system holding no prior at all.

Software. MIT-licensed, no required runtime dependencies, 218 tests. <https://github.com/AngshulMajumdar/BayesianRAG>

1 Introduction

An agent built on a language model interleaves generation with calls to external tools: retrievers, calculators, database queries, verification services. Execution is therefore a sequence of decisions under uncertainty in which the consequence of a choice is frequently unobservable until several steps later.

Mainstream frameworks do not treat it this way. Action selection is implemented as deterministic parsing of model output followed by a greedy choice, and failure is handled by retrying. These are the same decision seen twice: the agent maintains one trajectory. When a verification step contradicts an earlier retrieval, no alternative survives for that evidence to promote. Retrying does not repair this, because a retry samples afresh from the same proposal without conditioning on the intervening observation.

The consequences are ordinary and familiar. An agent commits to a fast, plausible, stale source and never revisits it. A retrieval pipeline returns the top-ranked passage when the answer sits at rank two. A pipeline that *does* validate discards the validated answer because it later looked somewhere else. Each is a failure of *inference*, not of the underlying tools.

1.1 Contributions

1. A formulation of agent execution as sequential inference over action trajectories, under which greedy selection, forward sampling, and SMC are inference strategies over one shared generative model (§2–§3).
2. A per-particle Beta–Bernoulli reliability model with retroactive credit assignment, letting a late verdict revise the standing of an earlier decision (§4).
3. An answer-selection rule ordered by validation status rather than recency, and a demonstration that the natural alternative silently discards confirmed answers (§5).
4. Marginalisation of posterior mass over *answers* rather than trajectories, yielding a consensus statistic and a principled criterion for requesting clarification (§6).
5. Controlled experiments isolating the contribution of inference, including results that do not favour the method (§8).

2 Agent Execution as Sequential Inference

2.1 Generative model

Let $a_t \in \mathcal{A}$ denote the action taken at step t (a tool invocation with arguments) and $o_t \in \mathcal{O}$ the resulting observation. Write the history $h_t = (a_{1:t-1}, o_{1:t-1})$, with $h_1 = \emptyset$. An episode of horizon T has joint density

$$p(a_{1:T}, o_{1:T}) = \prod_{t=1}^T \underbrace{p(a_t | h_t)}_{\text{proposal}} \underbrace{p(o_t | a_t, h_t)}_{\text{likelihood}}. \quad (1)$$

The proposal $p(a_t | h_t)$ is supplied by the model or policy: it is what a language model expresses when it emits a tool call. The likelihood $p(o_t | a_t, h_t)$ scores how well an observation coheres with the trajectory that produced it—high when a checker confirms a claim, low when it refutes one.

The quantity a practitioner wants is the posterior over trajectories given what the tools actually returned,

$$p(a_{1:T} | o_{1:T}) \propto \prod_{t=1}^T p(a_t | h_t) p(o_t | a_t, h_t). \quad (2)$$

Remark 1. Equation (1) is deliberately weak. It assumes only that actions are proposed conditional on history and that observations are scored conditional on the action and history. It does not assume Markov structure, a fixed tool set, or a differentiable policy.

2.2 Why greedy execution cannot recover

A conventional agent computes $\hat{a}_t = \arg \max_a p(a \mid h_t)$ and proceeds. This is maximum *a priori* selection: the likelihood $p(o_t \mid a_t, h_t)$ is never consulted. Because a single trajectory is retained, the posterior (2) is approximated by a degenerate measure at $\hat{a}_{1:T}$, and no observation can shift mass onto a branch that was never instantiated.

Proposition 1 (Greedy irrecoverability). *Let $\hat{a}_{1:T}$ be the greedy trajectory. For any observation sequence $o_{1:T}$, the greedy estimate is independent of $o_{1:T}$ except through its effect on the proposal $p(a_t \mid h_t)$. In particular, if the proposal at step t does not depend on o_{t-1} , then no evidence obtained at step $t - 1$ can alter the action taken at step t .*

This is why retrying does not help. A retry re-evaluates $p(a_t \mid h_t)$ with h_t largely unchanged and yields the same action, or a sample from the same distribution. It is a draw from the prior.

3 The Inference Ladder

Three behaviours arise as inference strategies over Equation (1). They differ in two binary properties: whether multiple hypotheses are instantiated (*explore*), and whether the observation likelihood is accumulated into their weights (*reweight*).

Regime	Particles	Explore	Reweight	Approximates
Deterministic greedy	1	no	no	point mass at the modal prior path
Probabilistic forward	N	yes	no	samples from the <i>prior</i> $p(a_{1:T})$
Bayesian SMC	N	yes	yes	the posterior $p(a_{1:T} \mid o_{1:T})$

Table 1: The inference ladder. Only SMC targets the posterior.

The distinction between the second and third rows is the one most easily conflated in practice: *maintaining several hypotheses is not the same as performing inference over them*. Forward sampling explores, but because weights are never revised, its particle cloud remains a sample from the prior. §8.1 shows this gap is large empirically—roughly a coin flip versus recovery in nine runs out of ten on the same tools.

3.1 Sequential Monte Carlo over trajectories

We approximate the posterior by a weighted particle set $\{(a_{1:t}^{(i)}, w_t^{(i)})\}_{i=1}^N$ with $\hat{p}(a_{1:t} \mid o_{1:t}) = \sum_i \bar{w}_t^{(i)} \delta_{a_{1:t}^{(i)}}$, where $\bar{w}_t^{(i)}$ are the normalised weights. At each step, particle i samples $a_t^{(i)} \sim q(\cdot \mid h_t^{(i)})$, executes it, and updates its weight by the incremental factor

$$w_t^{(i)} = w_{t-1}^{(i)} \cdot \frac{p(o_t^{(i)} \mid a_t^{(i)}, h_t^{(i)}) p(a_t^{(i)} \mid h_t^{(i)})}{q(a_t^{(i)} \mid h_t^{(i)})}. \quad (3)$$

Taking the proposal q equal to the model’s own action distribution $p(a_t \mid h_t)$ —the natural choice, since that is what the policy emits—the ratio collapses and Equation (3) reduces to

$$\log w_t^{(i)} = \log w_{t-1}^{(i)} + \log p(o_t^{(i)} \mid a_t^{(i)}, h_t^{(i)}), \quad (4)$$

accumulated in log space for numerical stability. All arithmetic uses the max-subtraction identity $\log \sum_i e^{x_i} = x^* + \log \sum_i e^{x_i - x^*}$, $x^* = \max_i x_i$, so weights spanning hundreds of orders of magnitude normalise without overflow.

3.2 Degeneracy and resampling

Particle weights concentrate over time. We monitor the effective sample size

$$\text{ESS}_t = \left(\sum_{i=1}^N (\bar{w}_t^{(i)})^2 \right)^{-1} \in [1, N], \quad (5)$$

and resample when ESS_t/N falls below a threshold τ (default 0.4), using systematic resampling—which has lower variance than multinomial resampling and $O(N)$ cost—after which weights are reset to uniform. Resampling is suppressed on the final step, where it would add variance without informing any subsequent decision.

3.3 The diversity requirement

Proposition 2 (Selector degeneracy). *If the selection rule is deterministic given a particle’s state, and all particles share an identical initial state, then all N particles follow identical trajectories. The weighted particle set is then a point mass, and reweighting has no effect: SMC reduces exactly to greedy execution at N times the cost.*

This is not a hypothetical. An arg max selector produces a cloud of identical particles, and the resulting system silently behaves as a slow greedy agent while appearing to run a particle filter. Selection must therefore sample in proportion to its score under exploring regimes. In the reference implementation the regime is passed to the selector precisely so this cannot be got wrong by accident, and a regression test asserts that greedy and SMC produce materially different trajectory distributions.

4 Tool Reliability and Retroactive Credit

4.1 Conjugate belief

Each tool j has an unknown success probability θ_j with a conjugate Beta prior. After k successes in n trials,

$$p(\theta_j \mid \mathcal{D}) = \text{Beta}(\alpha_j + k, \beta_j + n - k), \quad \mathbb{E}[\theta_j \mid \mathcal{D}] = \frac{\alpha_j + k}{\alpha_j + \beta_j + n}. \quad (6)$$

Beliefs are held *per particle*, not globally. A trajectory burned by an unreliable tool distrusts it, while a sibling that never invoked it does not—which is the correct semantics, since the two have observed different evidence.

Selection scores a candidate by the product of its posterior mean and a static *appeal* ρ_j encoding cost, latency, or convenience:

$$s_j = \mathbb{E}[\theta_j \mid \mathcal{D}] \cdot \rho_j. \quad (7)$$

Separating ρ_j from θ_j is deliberate. The failure mode this work addresses is precisely a tool that is *attractive and wrong*: high ρ , low θ . Collapsing them into one number makes that case inexpressible.

4.2 Retroactive credit assignment

The mechanism by which late evidence revises an earlier decision is that a scorer may emit belief updates against tools *other than* the one just executed. When a checker examines a claim and rejects it, the update is applied to whichever tool produced that claim:

$$\theta_{j^*} \leftarrow \text{Beta}(\alpha_{j^*}, \beta_{j^*} + \lambda), \quad j^* = \text{the tool that produced the examined claim}, \quad (8)$$

with λ a credit weight (default 2). A source is thus penalised when it is *caught*, not when it answers.

Remark 2 (Per-particle claim resolution). *A subtlety with real consequences: the proposer runs once per step and emits one candidate menu shared by all particles, but each particle must check its own claim. Binding the checker’s argument to any single particle’s claim causes cross-particle misattribution, under which a particle using a reliable tool is penalised for a claim produced elsewhere in the cloud— driving a correct tool’s posterior monotonically downward while it continues to be correct. The implementation resolves a **Pending CLAIM** placeholder per particle at execution time.*

4.3 Cross-episode transport

Beliefs may be carried between episodes with a tempering factor $\gamma \in [0, 1]$ applied to accumulated evidence:

$$(\alpha', \beta') = (\alpha_0 + \gamma(\alpha - \alpha_0), \beta_0 + \gamma(\beta - \beta_0)). \quad (9)$$

This transports the posterior itself. Reconstructing a Beta from its mean alone—the obvious shortcut—preserves the estimate while destroying the sample size behind it, converting fifty confident observations into a near-uninformative prior. Note that $\gamma < 1$ also shrinks the mean toward the prior, which is correct rather than incidental: with less evidence the estimate should be both less certain and closer to where it began.

5 Answer Selection Under Validation

Given a terminal particle, which of its observations constitutes the answer? The obvious rule—take the most recent—is wrong in a way that is easy to miss and expensive in practice.

Let $\mathcal{V}$ denote the set of claims a checker in this trajectory confirmed, and $\mathcal{R}$ the set it rejected. The correct preference order is

$$\text{answer} = \begin{cases} a \in \mathcal{V} & \text{if } \mathcal{V} \neq \emptyset, \\ \text{most recent } a \notin \mathcal{R} & \text{else if such } a \text{ exists,} \\ \text{most recent } a & \text{otherwise.} \end{cases} \quad (10)$$

Both refinements matter. Omitting the $\mathcal{R}$ clause lets an agent return a claim its own checker disproved—it gathered decisive evidence and then ignored it. Omitting the $\mathcal{V}$ clause is subtler: a trajectory that found the correct passage, validated it, and subsequently examined something else will return whatever it touched last, discarding a confirmation it had already paid for. Empirically this second error was worth between 3 and 35 percentage points depending on the task, and—because adding particles raises the chance that *some* trajectory happens to terminate on the validated answer—it masquerades convincingly as a budget limitation.

6 Posterior Consensus and Clarification

Trajectory mass is the wrong measure of agreement. Sixteen particles that reach one answer by sixteen different routes agree completely, yet each holds 1/16 of the mass. We therefore marginalise over answers,

$$\pi(a) = \sum_{i=1}^N \bar{w}_T^{(i)} \mathbb{I}[\text{answer}(a_{1:T}^{(i)}) = a], \quad \text{consensus} = \max_a \pi(a), \quad (11)$$

which distinguishes genuine disagreement from mere path diversity. When

$$\text{consensus} < \kappa \quad \text{and} \quad |\{a : \pi(a) > 0\}| > 1 \quad (12)$$

for a threshold κ (default 0.6), no answer commands enough of the posterior to be asserted, and the agent requests clarification rather than selecting arbitrarily among incompatible answers.

This is the state a single-path pipeline cannot represent. Having committed to one trajectory it holds exactly one answer and has no representation for the proposition that the evidence supports several. In our experiments the criterion fires in 13 of 200 runs where a checker settles the question, and in essentially all runs where two sources irreconcilably disagree—it asks when genuinely split, not by default.

7 Implementation

The reference implementation separates four concerns. A *proposer* emits candidate actions; a *selector* chooses among them per particle; an *environment* executes; a *scorer* maps observations to likelihoods and belief updates. A single runner implements

propose $\rightarrow$ select $\rightarrow$ observe $\rightarrow$ weight $\rightarrow$ resample if ESS collapses

for every regime; the regime supplies only a particle count and the two boolean flags of Table 1.

For the common case all three components are derived from tool metadata, so a working agent requires only tool declarations:

```
from bayesian_rag import Agent, tool, checker

@tool(reliability=0.6, appeal=0.9)      # attractive but unreliable
def quick_search(query: str) -> str:
    """Fast, sometimes stale."""
    ...

@tool(reliability=0.95, appeal=0.5)    # unglamorous but correct
def verified_search(query: str) -> str:
    """Slow, authoritative."""
    ...

@checker
def fact_check(text: str) -> bool:
    """Validate a claim."""
    ...

agent = Agent([quick_search, verified_search, fact_check])
result = agent.run("What is the capital of Australia?")

result.answer           # maximum a posteriori answer
result.consensus        # posterior mass backing it
result.status           # validated / unverified / refuted
result.needs_clarification # evidence insufficient to assert
result.reliability      # learned posterior means per tool
```

Listing 1: The complete high-level API.

Determinism and memoisation. Identical invocations are memoised within an episode, so N particles over a k -tool menu issue k calls per step rather than Nk . This is sound only for deterministic tools. A stochastic tool must be declared `deterministic=False`; otherwise every particle receives the same cached draw, the cloud collapses to a single sample, and the re-drawing mechanism that lets a filter exceed an unreliable tool’s accuracy is silently disabled. This was a real defect during development and is now covered by regression tests.

Reproducibility. Runs are exactly reproducible from a seed, and *only* from the seed: a test asserts that reseeding the global generator cannot perturb a seeded run, since a component reaching for global randomness breaks replication in a way that surfaces only when someone attempts to reproduce a result.

Statistics. All reported proportions carry Wilson score intervals, which remain within $[0, 1]$ and behave correctly near the boundaries—necessary here, since several conditions score exactly 0.000 or 1.000 where the normal approximation returns impossible bounds. Differences use Newcombe intervals and a two-proportion z -test. All are implemented in the standard library; the core package has no required runtime dependencies.

8 Experiments

All experiments use deterministic seeds indexed by trial, so every figure below regenerates exactly. Tools are synthetic, with i.i.d. Bernoulli success—an assumption we return to in §9.

8.1 The inference ladder

Two adversarial scenarios pit a cheap, confident, incorrect source against a slower authoritative one, with a checker that reveals the conflict one step later. A third scenario, with unambiguous routing, acts as a control. Crucially the proposer offers the same menu at every step: no recovery is scripted into the control flow, so any recovery observed is produced by inference.

Scenario	Greedy	Forward	SMC
<code>stale_vs_verified</code>	0.000	0.510	0.957
<code>web_vs_official</code>	0.000	0.520	0.947
<code>ambiguous_location</code> (control)	1.000	1.000	1.000

Table 2: Success rate, 300 seeds per cell. All contrasts on the adversarial scenarios are significant at $p < 10^{-10}$; none on the control are significant.

Greedy fails *completely*, not merely often: with one particle and an argmax selector it re-selects the attractive wrong source at every step, and no evidence can dislodge it. The forward-versus-SMC gap is the substantive result—both explore identically and differ only in whether the likelihood reaches the weights, and that alone converts a coin flip into recovery. The control confirms the machinery costs nothing where routing is already unambiguous.

8.2 Unreliable and adversarial tools

The regime where inference contributes most is one no fixed routing rule can address: tools that are correct *sometimes*, with no stable pattern. There is no correct branch to hard-code, because which source is right varies call to call.

The mechanism is independent re-drawing: each particle samples afresh, a checker validates, and the filter promotes whichever draw survives. It scales sharply—at 16 particles and 6 steps, a source correct 10% of the time yields a system correct 93.5% of the time; 64 particles convert a 35% source to 99%. A greedy agent cannot do this: one trajectory obtains one draw, with nothing to compare it against.

8.3 Prior sensitivity

The method inherits whatever reliability the practitioner declares. Against a tool with true accuracy 0.20:

Tool accuracies	LangGraph (engineered)	LangChain (reflexive)	SMC
one at 40%	0.607	0.407	0.933
two at 40%	0.607	0.607	0.887
30% + 90%	0.913	0.913	0.960
10% + 40% + 85%	0.820	0.820	0.867
three at 35%	0.567	0.567	0.787

Table 3: Stochastically unreliable tools, 150 seeds per cell.

Declared prior	0.2 (honest)	0.5	0.9	0.99 (confident lie)
SMC accuracy	0.993	0.887	0.527	0.380

A 61-point swing from the declaration alone. Inspection of the agent’s seeded selection scores identifies the mechanism: at a high declared prior the tool’s score (7) exceeds the checker’s, so the agent re-invokes the unreliable source *instead of validating it*. A confidently false prior does not merely misroute; it makes the agent stop checking its own work.

8.4 Comparison with deterministic orchestration

Results here do not uniformly favour the method, and we report them as measured. A naive pipeline is not an interesting baseline—it has no recovery path at all. The honest comparison is against a conditional graph written by someone who *knows* the failure mode: validate, and escalate on failure.

Across three worlds sharing one tool set—(A) the anticipated failure, (B) the same tools with roles reversed, (C) the escalation target unavailable—the engineered graph passes all three, at lower cost, deterministically. It passes (C) by crashing rather than reporting uncertainty, and its success in (B) is luck: its hard-coded preference happens to match reality. But it succeeds.

SMC with *honest* priors also passes all three; SMC with the declared (and, in world B, false) priors scores 0.31 and loses to every deterministic baseline. The conclusion we draw is narrow and worth stating plainly: use a conditional edge when the failure mode can be named in advance; reach for inference when it cannot—and only if the priors are honest, because the posterior faithfully propagates a confident lie.

Tool-call count is *not* reported as a finding. Maintaining a belief over alternatives costs more calls by construction; the question is what they buy.

8.5 Retrieval-augmented generation

We evaluate three RAG failure modes against three retrievers people actually deploy: BM25 (Okapi, $k_1 = 1.5$, $b = 0.75$), TF-IDF with cosine similarity, and LSA (TF-IDF with truncated SVD). Every instance is *verified* to defeat every retriever before its result is reported.

Instance	Normal RAG	Bayesian (BM25)	(TF-IDF)	(LSA)
1. Coreference	wrong, always	1.000	1.000	0.967
2. Staleness (vocab drift)	wrong, always	0.903	0.903	0.903
3. Noisy index	0.420	1.000	1.000	1.000

Table 4: 300 seeds, 95% Wilson intervals. Normal RAG is a single retrieval call, top-1, no checking.

Coreference. The answer-bearing passage refers to the entity as “the company” and never repeats its name, while a non-answering passage names it three times. BM25 scores them 3.06

versus 0.93; the answer lands at rank two. No lexical retriever performs coreference resolution. This is the most common real RAG failure—the *answer is in top-k but not top-1*—and we verify it is not a small-corpus artifact: the failure holds from 3 to 52 documents with the score gap widening.

Staleness under vocabulary drift. An outdated passage uses the canonical phrasing the query mirrors (“generating capacity”); the current amendment uses internal shorthand (“output rating”). BM25 scores them 5.40 versus 0.22. Lexical scoring has no notion of recency.

Index noise. The retriever is degraded to model replication lag. Normal RAG’s 0.420 matches the closed form $0.9 \times \frac{1}{3} + 0.1 \times 1.0 = 0.40$. The Bayesian pipeline declares the retrieval tool non-deterministic, so each particle draws independently—as when re-querying a replicated cluster.

9 Limitations

We state these plainly, and several arose from findings that contradicted our own earlier claims.

- **Synthetic tools.** All tools are mocks with i.i.d. Bernoulli success. Real failures correlate with query type, load, and upstream incidents in ways a fixed Bernoulli does not capture.
- **Perfect checkers.** Verifiers here compare against ground truth exactly. A real verifier is itself unreliable, which would compress every gap reported above. Two genuine checker bugs were found during this work; both are documented in the repository rather than quietly fixed.
- **Comparison baselines are semantic stand-ins,** not the real LangChain and LangGraph packages, which could not be installed in the evaluation environment. The engineered graph’s agreement with its closed-form expectation is partial evidence its control flow is correct. An adapter is provided to verify against the real packages.
- **Scenarios were constructed to separate the regimes.** Recovery is not scripted into the control flow, but the scenarios were designed to exhibit the failure modes of interest. They establish that the mechanism works where it matters, not how often that situation arises.
- **Priors are inherited, not learned from scratch.** A confidently wrong prior is costly, as §8 quantifies.
- **Short horizons and small corpora.** Episodes run to a few steps; corpora contain a handful of documents. Particle degeneracy over long horizons is not stressed, and BM25’s IDF statistics behave differently at scale.
- **Dense retrieval is only partially tested.** LSA falsified our own earlier conjecture that dense retrieval would resolve the coreference instance—it ranks the answer *worse* than BM25. Whether a pretrained sentence transformer would resolve it remains open.

10 Conclusion

Treating agent orchestration as inference over trajectories, rather than as control flow with retries, makes a specific and testable difference: evidence obtained late in an episode can revise a decision made early in it. Formulating greedy execution, forward sampling, and SMC as strategies over one generative model makes the comparison controlled, since the regime is the only thing that varies.

The empirical picture is not uniform, and we regard that as the more useful result. Where a failure mode can be named in advance, a conditional edge is cheaper, deterministic, and

sufficient. Where it cannot—unreliable sources, conflicting evidence, retrieval whose ranking is wrong in ways not known beforehand—maintaining a posterior over alternatives converts sources that are individually unreliable into systems that are collectively dependable, and supplies a calibrated signal for when to stop guessing and ask.

Availability

Software, experiments, figures, and raw results: <https://github.com/AngshulMajumdar/BayesianRAG>. MIT licensed. The core package has no required runtime dependencies. Every table in this paper is regenerated by a documented command.

References

- [1] A. Doucet and A. M. Johansen. A tutorial on particle filtering and smoothing: fifteen years later. *Handbook of Nonlinear Filtering*, Oxford University Press, 2009.
- [2] N. Chopin and O. Papaspiliopoulos. *An Introduction to Sequential Monte Carlo*. Springer, 2020.
- [3] P. Del Moral, A. Doucet, and A. Jasra. Sequential Monte Carlo samplers. *Journal of the Royal Statistical Society: Series B*, 68(3):411–436, 2006.
- [4] A. Kong, J. S. Liu, and W. H. Wong. Sequential imputations and Bayesian missing data problems. *Journal of the American Statistical Association*, 89(425):278–288, 1994.
- [5] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [6] R. G. Newcombe. Interval estimation for the difference between independent proportions. *Statistics in Medicine*, 17(8):873–890, 1998.
- [7] W. R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3–4):285–294, 1933.
- [8] T. Lattimore and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.
- [9] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.
- [10] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 33:9459–9474, 2020.
- [11] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023.
- [12] T. Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 36, 2023.
- [13] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [14] S. Deerwester et al. Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41(6):391–407, 1990.